Shared Project Understanding — Remote Patient Vital Signs Monitoring System

Purpose of this document: a common, source-based explanation for all five team members before the HOD individual discussions and viva. It describes what the repository supports today; it is not proof of personal authorship, clinical validity or physical sensor performance.

Authorship note: member names are deliberately replaced by `[MEMBER 1 NAME]` … `[MEMBER 5 NAME]`. The team must confirm who performed each task. Unless the team has direct evidence, say **"the team implemented"** rather than making a personal authorship claim for a shared file.

0. Document control

Item	Value
Project	Remote Patient Vital Signs Monitoring System / RPM IoMT Patient Monitoring Node
Repository	`RPMIOMTproject`
Branch / HEAD	`main` / `f67c13a`
Firmware identity	`rpm-2.1.0`, telemetry schema `2`
Preparation date	2026-10-29
Audit records	`CODE_AUDIT_MANIFEST.md`, `CODE_CLEANUP_REGISTER.md`, `NEXT_STEPS.md`
Scope	Engineering prototype: firmware, local serial bridge, SQLite history, dashboard, baseline and deterministic alerting
Excluded from validation	Clinical diagnosis, emergency response, sensor accuracy, regulatory approval, real-patient deployment

The working tree already contained uncommitted repair and integration changes before this documentation set was created. These documents do not modify production source and do not claim that this documentation work made those repairs.

1. Project in one paragraph

The project is an ESP32-S3-based remote-monitoring prototype. Installed sensors measure optical pulse/SpO2 estimates (MAX30102), motion/fall candidates (MPU-6050 or MPU-6500) and the actual temperature of a TMP117 sensor package. The firmware publishes one-line JSON telemetry over USB serial at 115200 baud with a [JSON] prefix. A local Python bridge validates that telemetry, serves a browser dashboard over Server-Sent Events (SSE), stores valid observations in a local SQLite database, maintains a per-wearer seven-day baseline, and records SOS, fall and baseline-deviation events. The design deliberately separates the **actual TMP117 sensor temperature** from the **clinical wrist-skin temperature**, which remains unavailable because wrist contact is not verified. There is no core-temperature estimate and no fever diagnosis.

2. Problem, users and purpose

2.1 Problem being addressed

A caregiver needs a compact local view of a wearer's vital signs, motion state and environmental context, with a history rather than only a live number. The prototype explores how to collect multi-sensor telemetry honestly: missing or untrusted values should be visibly unavailable, not silently replaced by zero or by a plausible-looking number.

2.2 Intended demonstration users

- **Wearer/patient (simulated):** provides the device, fits the sensors and can press the SOS button.
- **Caregiver/clinician (demonstration audience):** reads the dashboard, reviews history and baseline context.
- **HOD/examiner:** checks whether the team understands the engineering, data semantics, evidence limits and division of work.

2.3 Objectives implemented in software

1. Initialize the installed I2C devices and report which are online.
2. Acquire heart-rate, SpO2, motion, actual TMP117 temperature and SOS/fall state.
3. Represent validity, warming, low quality, staleness, sensor error and unavailability without fabricating values.

4. Publish a documented JSON contract every second, plus immediate SOS/fall packets.

5. Validate the contract strictly in the bridge before accepting data.

6. Serve a live dashboard and a gap-preserving local history.

7. Create a per-wearer baseline only after sufficient data exists.

8. Provide a deterministic threshold/alert view without pretending it is a trained diagnostic model.

2.4 Explicit non-goals of the current build

- Diagnosing fever, sepsis, hypoxia, arrhythmia or any disease.
- Estimating core body temperature by adding an offset to wrist or TMP117 temperature.
- Replacing a clinical thermometer, pulse oximeter or emergency service.
- Claiming sensor accuracy without reference measurements.
- Claiming SHT40 or MLX90614 operation: neither device is installed.
- Claiming a trained machine-learning model or an integrated LLM: neither exists.

3. What is genuinely notable — and what is not

3.1 The engineering contribution

The strongest contribution is **integration with conservative data semantics**:

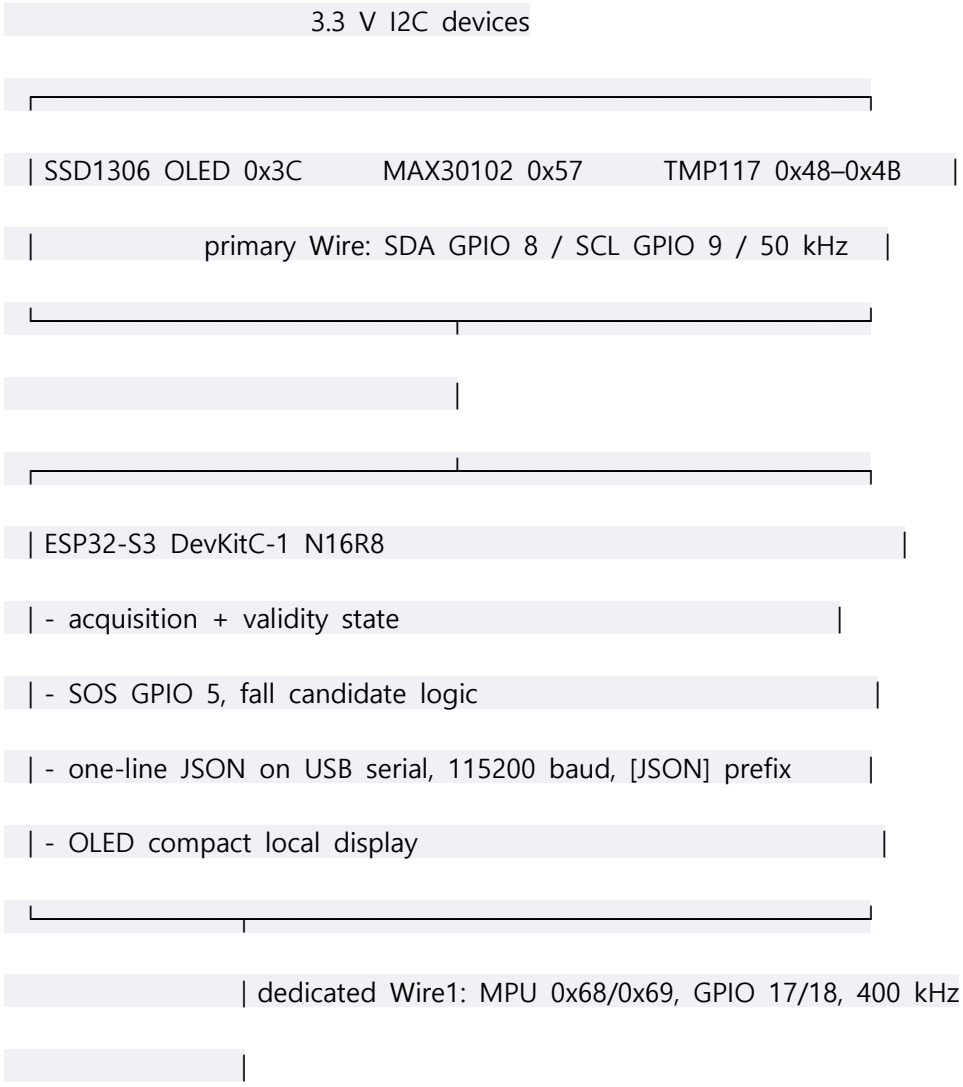
- one firmware contract ties several sensors into a single telemetry packet;
- the bridge rejects contradictory or unsafe packets instead of displaying them;
- the clinical wrist value, the actual sensor temperature and core temperature are separate concepts;
- missing data produces `null` and a visible status, not a substituted zero;
- history preserves disconnection gaps;
- the dashboard repeats the same safety rules as the backend;
- the tests target the boundary cases that caused earlier conflation.

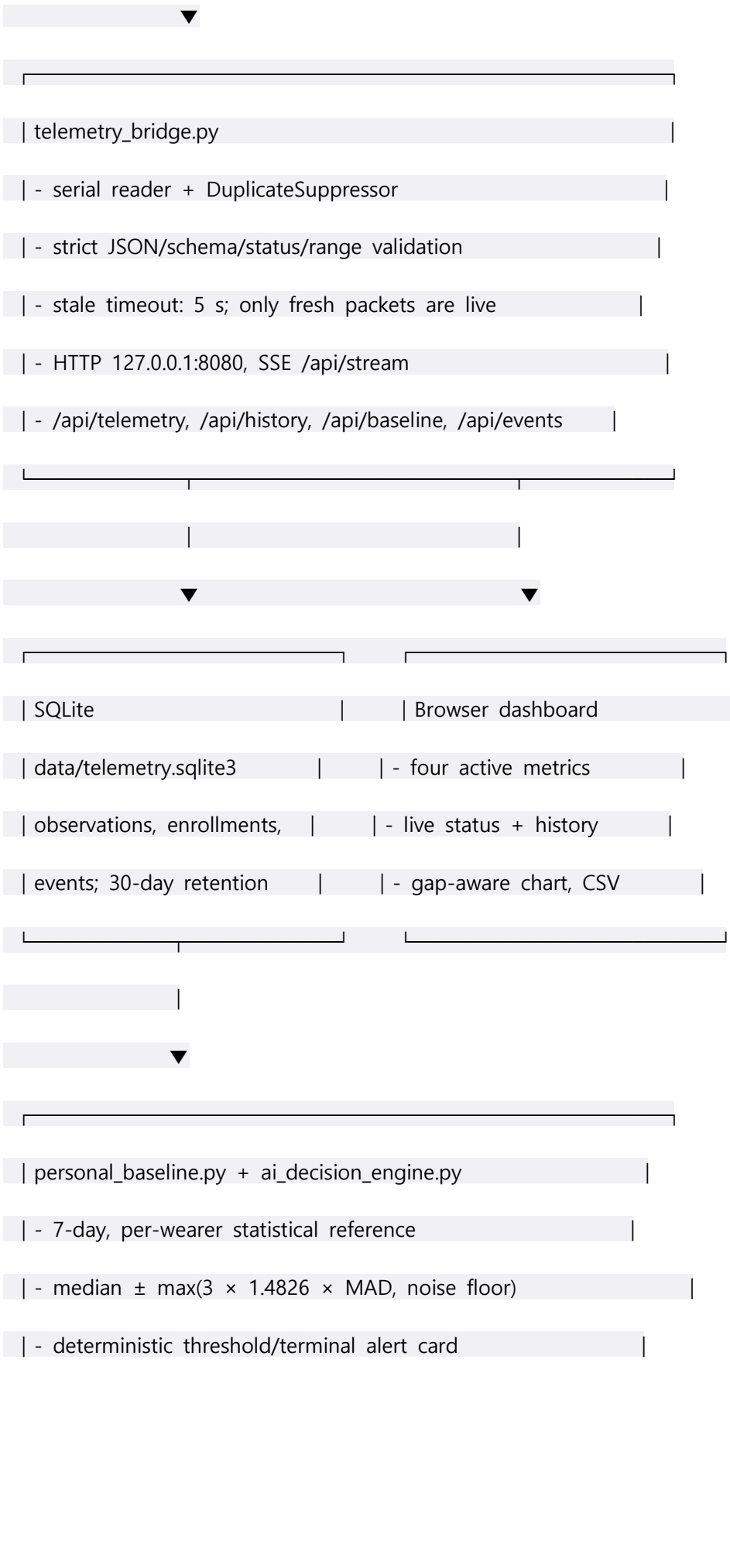
3.2 What is not novel

- The individual sensors are commercial off-the-shelf modules.
- The optical HR/SpO2 calculation is library-provided processing, not a new clinical algorithm.
- The baseline is median/MAD statistics, not a trained model.
- The alert engine is deterministic rule evaluation.
- The system is not clinically validated.

Use this sentence in the viva: **"The novelty claim is limited to the integration and the fail-closed temperature semantics of this prototype; it is not a new diagnostic method."**

4. System architecture




▼
telemetry_bridge.py
- serial reader + DuplicateSuppressor
- strict JSON/schema/status/range validation
- stale timeout: 5 s; only fresh packets are live
- HTTP 127.0.0.1:8080, SSE /api/stream
- /api/telemetry, /api/history, /api/baseline, /api/events
▼
▼
SQLite
data/telemetry.sqlite3
observations, enrollments,
events; 30-day retention
Browser dashboard
- four active metrics
- live status + history
- gap-aware chart, CSV
▼
personal_baseline.py + ai_decision_engine.py
- 7-day, per-wearer statistical reference
- median ± max(3 × 1.4826 × MAD, noise floor)
- deterministic threshold/terminal alert card

The optional SHT40 and MLX90614 branches are compiled out by default with RPM_ENABLE_OPTIONAL_SHT40=0 and RPM_ENABLE_OPTIONAL_MLX90614=0. They are not part of the installed inventory and must not be drawn as working sensors.

5. Installed hardware

Component	Bus / address	Purpose	Current evidence
ESP32-S3 DevKitC-1 N16R8	16 MB QIO flash, 8 MB OPI PSRAM	Controller and telemetry producer	Build target; board metadata only
SSD1306 128×64 OLED	Wire, 0x3C, SDA 8 / SCL 9	Local display	Firmware path exists; physical rendering unverified
MAX30102	Wire, 0x57, SDA 8 / SCL 9	Optical pulse and SpO2 estimates	Software checks exist; accuracy/placement unverified
7SEMI TMP117 ES-12132	Wire, one of 0x48–0x4B, SDA 8 / SCL 9	Actual sensor-package temperature	Driver and raw diagnostic exist; physical identity/raw recording pending
MPU-6050 / MPU-6500	Wire1, 0x68 or 0x69, SDA 17 / SCL 18	Acceleration, gyroscope, motion/fall candidate	Software path exists; physical axis/range/calibration tests pending
SOS pushbutton	GPIO 5 to GND, active LOW	Emergency event	Firmware path exists; physical press test pending
SHT40	Not installed	—	Compiled out; not counted; not tested
MLX90614	Not installed	—	Compiled out; not counted; not tested

Primary Wire is configured at 50 kHz; dedicated IMU Wire1 at 400 kHz. All breakouts are treated as 3.3 V devices. sensors.active_max is fixed at **3** because the installed measurement inventory is max, mpu, tmp117. The OLED is a display, not a measurement sensor.

6. Software modules

File / area	Responsibility	What it contains
src/main.cpp	Firmware and integration	I2C helpers, sensor init/acquisition, validity states, SOS/fall, JSON, OLED, hot-plug retry
src/tmp117_protocol.h	TMP117 protocol	Register addresses, 0x0117 identity, 0x01A0 configuration, signed conversion, stuck/range checks, compile-time vectors
src/sensor_math.h	Sensor arithmetic	MPU 4096 LSB/g and 131 LSB/dps; MAX signal-quality thresholds and perfusion index
diagnostics/tmp117_raw/	Independent diagnostic	Scans 0x48–0x4B, checks WHO_AM_I, config readback, offset, raw bytes and conversion
telemetry_bridge.py	Serial/HTTP backend	Strict packet parsing, duplicate handling, stale detection, SSE, APIs, event recording
telemetry_store.py	SQLite history	Valid-observation selection, additive migration, wrist evidence checks, 30-day retention, gap-preserving buckets
temperature_safety.py	Temperature semantics	Actual versus clinical wrist separation, raw conversion checks, core/fever policy, exact message
personal_baseline.py	Seven-day baseline	Per-wearer enrollment, qualified days/minutes, median/MAD model freeze, deviation events
ai_decision_engine.py	Deterministic clinical summary	Validated field extraction, threshold findings, terminal alert card; no ML/LLM

File / area	Responsibility	What it contains
index.html, assets/dashboard.js, assets/dashboard_core.js, assets/dashboard.css	Dashboard	Four active metric cards, page views, SSE/history rendering, separate temperature labels, CSV, gaps
tests/	Host evidence	Synthetic Python and Node fixtures and boundary tests; no clinical data and no hardware proof
README.md, wiring-guide.html	Operating documentation	Hardware, policy, telemetry, verification procedures, limitations

7. End-to-end data flow

1. **Boot:** setup() starts serial at 115200, configures Wire and Wire1, scans I2C, initializes OLED, MPU, MAX30102 and TMP117, then emits the first packet.

2. **Acquisition:** loop() services the SOS button, reads the MPU every 20 ms, samples MAX30102, reads TMP117 every 2 s (500 ms conversion cycle plus averaging), and refreshes statuses.

3. **Validity:** each sensor path sets a status such as valid, warming_up, low_quality, stale, sensor_error or not_available. Missing/stale values are serialized as null.

4. **Transmission:** outputTelemetry() writes one line beginning [JSON] {...}. It runs once per second and immediately for SOS/fall. refreshSensorStatuses() is called first even on urgent paths so stale data cannot be published as valid.

5. **Parsing:** telemetry_bridge.py reads complete lines, suppresses identical repeats within 1.5 s, rejects duplicate JSON keys, then calls parse_packet() → _validate_packet().

6. **Acceptance:** only a valid packet updates latest_telemetry; TelemetryStore.record() and PersonalBaseline.compare() receive it; sensor/fever/wrist evidence is checked again.

7. **Live view:** SSE /api/stream sends snapshot() data. A packet older than 5 s is not served as live; a new browser is never shown a stale last packet.

8. **History:** /api/history aggregates valid SQLite observations into buckets and fills missing buckets with nulls.

9. **Dashboard:** consume() updates the live cards; drawChart() uses DashboardCore.gapAwarePath() so a line is not drawn across a gap.

10. **Baseline:** /api/baseline creates or reports a per-wearer enrollment; when 7 days and at least 30 sampled minutes per day are available for a metric, a frozen median/MAD reference is stored.

11. **Alerts:** SOS/fall are firmware booleans; optical thresholds are deterministic; baseline departures require three consecutive outside readings and are rate-limited to one event per metric per 300 s.

8. Telemetry contract — schema 2

The firmware emits a single JSON object per line at 115200 baud, prefixed with [JSON]. The bridge is strict about the fields it consumes and rejects many contradictions.

8.1 Top-level fields

uptime_s, device_id, firmware_version, schema_version, board, risk, urgent, sensors, sensor_health, vitals, alerts, status_summary.

The firmware's current device_id is RPM-NODE-01; firmware_version is rpm-2.1.0; schema_version is 2.

8.2 Sensors

```
"sensors": {
  "oled": true,
  "max": true,
  "mpu": true,
  "tmp117": true,
  "sht40": false,
  "mlx": false,
  "active": 3,
  "active_max": 3
```

}

active is the number of present installed measurement sensors among max, mpu, tmp117. active_max is always 3. The bridge rejects sht40: true, mlx: true, a wrong active_max, or an active count that contradicts the three booleans.

8.3 Important vital fields

Optical: heart_rate, heart_rate_status, heart_rate_timestamp_ms, spo2, spo2_status, spo2_timestamp_ms, finger_on, raw_ir, raw_red, optical_signal_quality, optical_perfusion_index.

Temperature: tmp117_sensor_temperature_c, tmp117_sensor_temperature_status, tmp117_sensor_temperature_raw, tmp117_sensor_temperature_timestamp_ms, wrist_contact_status, wrist_skin_temperature_c, wrist_skin_temperature_status, wrist_skin_temperature_raw, wrist_skin_temperature_timestamp_ms, estimated_core_temperature_c, estimated_core_temperature_status, fever_screening.

Motion: acceleration, gyroscope, imu_raw, motion_g, motion_status, motion_timestamp_ms, activity_state, orientation, steps.

Legacy fields: temp_core, temp_skin, temp_object, temp_amb, sht40_temp_c, sht40_humidity_pct may be absent or null; non-null values are rejected by the bridge because those modules are not installed. The firmware does not emit tmp117_temp_c; the bridge tolerates only a null/absent legacy alias.

Alerts: {"sos": false, "fall": false}.

8.4 Status rule

For a valid value, the corresponding status must be valid, the sensor must be present, the value must be in the configured range, and required timestamp/raw evidence must be present. For a non-valid status, the value (and, where applicable, raw/timestamp) must be null; a non-null value is rejected. Missing, stale, warming, low-quality and sensor-error values are never stored as zero or as a previous reading.

9. Exact temperature semantics

This is the most important shared concept. Do not collapse the three temperatures into one number.

Concept	JSON field	Meaning	Current build	Accepted range in code

Actual TMP117 sensor temperature	tmp117_sensor_temperature_c	The actual TMP117 package/sensor reading; diagnostic telemetry	Emitted when the TMP117 read is valid	-55 °C to 150 °C
Clinical wrist-skin temperature	wrist_skin_temperature_c	A peripheral wrist measurement, only meaningful with verified contact and matching evidence	Always null; contact/status always unverified	-10 °C to 60 °C when a future verified-contact method exists
Core temperature	estimated_core_temperature_c	A core-body estimate	Always null; status not_available	Not implemented
Fever screening	fever_screening	A clinical classification from a validated method	not_available; message exact below	Not implemented

9.1 Rules that must be stated correctly

- The TMP117 reading is **actual sensor temperature**, not wrist skin temperature and not core temperature.
- tmp117_sensor_temperature_c may be valid outside the clinical wrist range but inside -55..150 °C; it must never be promoted to a clinical wrist value.
- The current firmware always writes wrist_contact_status: "unverified", wrist_skin_temperature_c: null, wrist_skin_temperature_status: "unverified", wrist_skin_temperature_raw: null, and wrist_skin_temperature_timestamp_ms: null. It does this because no validated contact detector is installed or verified.
- A future verified wrist value would require all of the following: wrist_contact_status: "verified", status valid, raw evidence, timestamp, signed conversion within 0.02 °C, actual-sensor value/raw/timestamp matching, and the wrist value within -10..60 °C. Until then, no clinical value is produced.
- No fixed offset or conversion from wrist to core is applied. There is no core = wrist + k

rule.

- Core remains unavailable even if the actual TMP117 sensor and a future wrist value are both valid. A wrist reading cannot confirm or rule out fever.
- The exact message is: **Core temperature unavailable — confirmation required**.
- The OLED is limited by its 128×64 display and built-in font; it uses compact `TMP:` and `Wrist:` text, and shows `Core temp unavailable` / `confirmation required` on two lines because the font lacks the em dash. The required full labels and exact message are implemented in the dashboard and JSON, not necessarily rendered character-for-character on the OLED. Do not treat the OLED shorthand as the clinical label policy.
- Missing, stale, warming, low-quality, sensor-error, `not_available` and `unverified` values are `null`; the bridge rejects a non-null value with a non-valid status.

9.2 TMP117 raw conversion and safety checks

`tmp117_protocol.h` and the firmware use a signed 16-bit big-endian code:

`signed = raw >= 0x8000 ? raw - 0x10000 : raw`

`temperature_C = signed × 0.0078125`

Examples: `0x0ED3` → `29.6484375 °C`; `0xFFFF` → `-0.0078125 °C`; `0x0000` → `0.0 °C`. A two-decimal display would round `0xFFFF` to `-0.01 °C`.

The firmware:

- scans addresses `0x48–0x4B` and requires `WHO_AM_I (0x0F) == 0x0117`;
- writes configuration `0x01A0` (continuous conversion, 8-sample averaging, 500 ms cycle) and checks the writable-mask readback;
- reads configuration before temperature because reading configuration clears `DATA_READY`;
- rejects raw `0x0000` and `0xFFFF` as stuck-bus patterns;
- rejects values outside `-55..150 °C`;
- requires one confirming sample when a new reading jumps more than `5 °C` from the last accepted reading; a second reading within `0.5 °C` allows the new value;
- applies no software offset; a nonzero hardware offset is reported but not silently corrected.

9.3 What “fever unavailable” means

The fever_screening object has status: "not_available", rule: null, a null/unmeasured wrist input, the uncertainty statement, limitations, and the exact message. A wrist reading alone cannot confirm or rule out fever. The system does not diagnose; the recommended action is confirmation with a validated clinical thermometer and clinician assessment.

10. Backend behavior

10.1 Strict parsing and acceptance

- Serial lines are read at 115200; the bridge handles partial reads with a buffer and splits on newline.
- Lines that do not begin with { after removing [JSON] are ignored.
- _object_without_duplicates() rejects duplicate JSON keys; this is separate from DuplicateSuppressor, which drops identical serial lines within 1.5 s.
- _validate_packet() checks types, finite numbers, schema 2, sensor booleans, active count, status/value consistency, measured ranges, timestamp rules, temperature evidence, IMU evidence and fever policy.
- Invalid packets increment invalid_packets; they do not update the live state or history.
- snapshot() marks the node disconnected after 5 s without a fresh packet and returns telemetry: null; a new browser does not receive a stale packet.
- _accept_packet() applies a server receive timestamp _received_at, updates the live state, records valid observations, and computes baseline comparisons.

10.2 Local APIs

The bridge binds only to 127.0.0.1, default port 8080:

Route	Method	Purpose
/api/telemetry	GET	Latest hardware status and fresh telemetry snapshot
/api/stream	GET/SSE	Live telemetry/status; per-client queue of 8, slow clients drop oldest

Route	Method	Purpose
/api/history?seconds=…&device_id=…	GET	Bucketed valid history with null gaps; seconds clamped to 60–604800
/api/baseline?device_id=…	GET	Baseline summary, enrollment state and per-metric models
/api/baseline	POST	Start/end collection; JSON body, size 1–4096 bytes; same-origin check; connected current device required
/api/events?device_id=…	GET	Up to 30 most recent events
/, /index.html, /wiring-guide.html, /assets/…	GET	Static dashboard assets

10.3 SQLite history

- Database default: data/telemetry.sqlite3 (local and excluded from source control; it stores device IDs and a baseline enrollment name/alias, so real patient identifiers must not be entered in a demo).
- Tables: observations, enrollments, events; WAL mode and a device/time index.
- observations stores valid values plus status context and raw wrist code where valid; invalid/missing readings are NULL.
- _migrate() adds columns to older databases rather than rewriting them.
- Records older than 30 days are pruned at most once per hour.
- History bucket size is max(1, seconds // 450); missing buckets are filled as null points, so the chart can break the line at a gap.
- The actual TMP117 sensor temperature is not stored as the clinical wrist metric.

10.4 Event behavior

SOS and fall events are inserted when the alert becomes newly active. A baseline deviation is inserted after three consecutive outside readings within 5 s each, with a 300 s cooldown per metric. Event detail is human-readable and contains no diagnosis.

11. Dashboard and storage behavior

- Pages: overview, baseline, device/sensors.
- Active metric cards: heart rate, SpO2, **Wrist Skin Temperature (°C)**, movement. The actual sensor value is shown separately as **TMP117 Sensor Temperature (°C)** in the temperature environment area.
- The clinical wrist card shows no numeric value while contact is unverified; raw/timestamp evidence stays hidden.
- The exact core message **Core temperature unavailable — confirmation required** is displayed; the card does not invent a core number.
- No SHT40 or MLX90614 tabs/cards exist.
- The history chart uses `DashboardCore.gapAwarePath()`; missing buckets start a new segment rather than drawing through the gap.
- CSV export contains active metrics; missing values are blank.
- `DashboardCore.metricStatus()` rechecks sensor presence, status, raw conversion, ranges and matching actual/wrist evidence, independent of the backend.
- `DashboardCore.temperatureAssessment()` always returns `estimated_core_temperature_c: null`, `estimated_core_status: "not_available"`, and fever `not_available`.
- HTML output is escaped in the dashboard code where dynamic text is inserted.

12. Baseline and deterministic alerting

12.1 Personal baseline

- `PersonalBaseline` creates one active enrollment per device and a new wearer gets an empty model set.
- Qualification requires **7 days** with at least **30 distinct sampled minutes per day** for a metric, plus elapsed time since the first minute of at least 7 days.
- Data are reduced to minute means before statistics.
- Model: `center = median(values)`, `width = max(3 × 1.4826 × MAD, noise floor)`, `low = center - width`, `high = center + width`.

- Noise floors: heart rate 5.0 bpm, SpO2 2.0 %, wrist skin temperature 0.2 °C, motion 0.1 g.
- Models freeze once created; they are not retrained on every packet.
- Three consecutive readings outside the frozen range, with no more than 5 s between consecutive readings, create a deviation event; events are limited to one per metric per 300 s.
- The baseline requires all four active metrics to reach ready state for overall ready. While wrist contact is unverified, wrist values are not stored, so the overall baseline cannot qualify; this is intentional, not a fabricated data path.
- Wrist skin temperature is baseline context only, never core temperature.

12.2 Deterministic alert engine

ai_decision_engine.py is named "AI" but currently contains deterministic logic. It:

- extracts only values whose sensor is present and status is valid;
- treats missing optical values as UNAVAILABLE, not zero;
- marks RED (EMERGENCY) for SOS/fall;
- marks YELLOW (WARNING) for heart rate < 50 or > 120 bpm, or SpO2 < 90 %;
- marks GREEN (NO OPTICAL ALERT) only when both HR and SpO2 are valid;
- otherwise reports UNKNOWN (MISSING DATA);
- always reports actual TMP117 temperature as actual sensor telemetry, not clinical wrist/core;
- always includes the exact fever message and recommends clinical confirmation if fever is suspected.

There is **no trained model, no neural network and no LLM integration**. "AI" is the legacy module name, not evidence of AI.

13. Evidence classification

Use these tags consistently in the viva. They prevent software claims from turning into hardware or clinical claims.

Tag	Meaning	Examples
[SOURCE]	Implemented and visible in the repository	parse_packet(), readTmp117(), DashboardCore.gapAwarePath()
[TEST]	Automated host test passed in this documentation pass	62 Python tests, 10 Node tests; TMP117 vectors, schema rejection, history gaps
[BUILD]	PlatformIO compile/link succeeded	main firmware and TMP117 diagnostic
[RECORDED HW OBS]	Previously observed on hardware, but not reproduced in this pass	raw 0x0ED3 → 29.6484375 °C; diagnostic 0x0DA7 → 27.30469 °C
[UNVERIFIED HW]	Source/build exists, but physical behavior is not verified	TMP117 identity/config on the actual board, MAX30102 accuracy, MPU axes/calibration, OLED rendering, SOS/fall on hardware
[PLAN]	Documented future work, not current behavior	verified contact method, core-temperature method, CI, clinical validation

13.1 Fresh verification in this documentation pass (2026-10-29)

Check	Result
python -m pytest -q	**62 passed**
node --test tests/dashboard_core.test.js	**10 passed**
Ruff, Python compilation, JavaScript syntax	**passed** (combined check reported all checks passed)
pio run -e esp32-s3-devkitc-1	**SUCCESS**, RAM 21,424 / 327,680 bytes, flash 326,001 / 6,553,600 bytes
pio run -d diagnostics/tmp117_raw	**SUCCESS**, RAM 18,916 / 327,680 bytes, flash 282,377 / 6,553,600 bytes

These are host/build results. They do **not** prove sensor accuracy, contact verification, clinical validity or emergency reliability.

13.2 Prior hardware observations — preserve the label

The audit records contain these earlier observations, which were not reproduced in this pass:

- raw `0x0ED3 → 29.6484375 °C`;
- raw `0xFFFF → -0.0078125 °C`, which a two-decimal display rounds to `-0.01 °C`;
- diagnostic raw `0x0DA7 → 27.30469 °C`;
- 60 reads, 0 failures, 0 stuck patterns;
- TMP117 was reported at address `0x49` in a prior observation, while the driver scans `0x48–0x4B`.

Correct label: **HARDWARE VERIFICATION REQUIRED — prior observation only, not reproduced in this audit.** Do not present these values as a live measurement or as accuracy evidence.

13.3 Not performed in this documentation pass

- No firmware upload or serial capture.
- No reference-thermometer comparison.
- No wrist-contact verification.
- No physically assembled end-to-end device test.
- No MAX30102 accuracy/placement test.
- No MPU axis/range/fall test.
- No OLED visual inspection.
- No SHT40 or MLX90614 test (neither is installed).

14. Common misconceptions and corrected answers

1. **"The TMP117 is the wrist thermometer."** No. It measures its own package/sensor temperature. The clinical wrist metric is separate and currently unavailable.
2. **"Unverified means the sensor is broken."** No. `wrist_contact_status: "unverified"` means contact is not verified. The actual TMP117 sensor can still be valid.

3. **"A missing value is zero."** No. Missing/invalid/stale values are JSON `null`; tests explicitly distinguish zero from missing.

4. **"We have four measurement sensors."** No. Installed measurement inventory is MAX30102, MPU and TMP117 = 3; OLED is a display; SHT40/MLX90614 absent.

5. **"The system estimates core temperature."** No. Core is always null/`not_available`; fever screening is unavailable.

6. **"The system uses AI/LLM."** No. The alert engine is deterministic; the module name is legacy.

7. **"The tests prove the sensors work."** No. They are synthetic host/protocol tests. Hardware verification remains required.

8. **"SHT40/MLX90614 data is available."** No. Both are not installed, compiled out and rejected if claimed present.

9. **"A fixed offset turns wrist into core."** Rejected. No offset is applied; wrist is peripheral context only.

10. **"The OLED uses the exact clinical labels."** The OLED uses compact text because of its 128×64 display; exact labels are implemented in JSON/dashboard. OLED rendering is unverified.

11. **"This is an emergency-response device."** No. SOS/fall are prototype alerts; there is no validated dispatch, acknowledgement or clinical service.

12. **"The baseline is a diagnostic model."** No. It is a per-wearer median/MAD statistical reference with engineering noise floors.

15. Five-member responsibility map (summary)

Details and handoffs are in `TEAM_RESPONSIBILITY_MATRIX.md`. This is a proposed division to explain, not proof of authorship.

Member	Primary responsibility	Must be able to defend
[MEMBER 1 NAME]	Firmware and system integration	src/main.cpp, scheduling, status propagation, JSON/OLED, SOS/fall, build
[MEMBER 2 NAME]	Hardware and sensor engineering	wiring, TMP117 protocol/diagnostic, MAX

Member	Primary responsibility	Must be able to defend
		quality guards, MPU conversion/calibration limits
[MEMBER 3 NAME]	Communication and backend	serial reader, strict validation, stale handling, SSE/APIs, storage handoff
[MEMBER 4 NAME]	Dashboard and storage	exact labels, null/gap rendering, SQLite history/CSV, accessibility, backend contract
[MEMBER 5 NAME]	Analytics, safety and quality	temperature policy, baseline statistics, deterministic alerting, test/audit evidence

All five must know the temperature semantics, null policy, installed inventory and evidence limits, because those cross every module.

16. Current limitations and future work

16.1 Limitations

- No physical sensor validation; no reference thermometer or optical reference.
- Wrist contact is not verified, so clinical wrist and baseline temperature cannot qualify.
- Core temperature and fever screening are deliberately unavailable.
- TMP117 prior `-0.01 °C` anomaly still needs a raw diagnostic log to confirm/refute the `0xFFFF` stuck-bus hypothesis.
- MPU calibration is only a startup bias/single-orientation estimate, not full six-face calibration.
- Fall detection is a heuristic, not a validated fall detector.
- Optical SpO2/HR are estimates; placement and signal quality matter.
- Bridge is localhost-only, with no authentication/TLS; not suitable for open network or real patient data.
- SQLite and the dashboard are a local prototype, not a regulated record system.
- No CI yet; checks were run manually.

● No regulatory approval or clinical study.

16.2 Ordered future work

1. Preserve the exact current source/test/build evidence; review the uncommitted diff before any release decision.
2. Run the TMP117 raw diagnostic on the assembled board for at least five minutes; record address, WHO_AM_I, config readback, offset, raw patterns, failures and DATA_READY.
3. Compare actual TMP117 readings with a validated reference under controlled conditions; document uncertainty. Do not add a fixed offset.
4. Define and validate a wrist-contact verification method before exposing clinical wrist values.
5. Validate MAX30102 HR/SpO2 and MPU axes/ranges/calibration/fall behavior physically.
6. Decide whether a validated core-temperature method will ever be added; until then keep core null and fever unavailable.
7. Add CI for Python, Node and both PlatformIO builds.
8. If the prototype is ever networked, add authentication, TLS, privacy review and a proper clinical/regulatory process.

17. Glossary

Term	Meaning
TMP117	Digital temperature sensor; here its reading is actual sensor-package temperature.
Wrist skin temperature	Peripheral clinical metric that requires verified wrist contact; not core temperature.
Core temperature	Internal body temperature; not measured or estimated in this build.
Fever screening	Clinical classification; unavailable because no validated method/reference exists.
unverified	Contact-status value meaning wrist contact has not been verified; not a sensor-error synonym.
null	Explicit JSON absence; never a substitute zero or stale value.

Term	Meaning
active_max	Fixed installed measurement-sensor denominator, currently 3.
Baseline	Per-wearer seven-day median/MAD reference, not a diagnostic model.
Gap-aware chart	Chart that breaks its line where a history bucket has no valid data.
Stuck-bus pattern	TMP117 raw 0x0000/0xFFFF treated as untrustworthy, not as real temperatures.
SSE	Server-Sent Events used by /api/stream for live dashboard updates.
Deterministic alert	Rule-based threshold/event logic; no trained model.

18. Source map for quick reference

Question	Start here
How is the firmware structured?	src/main.cpp: setup(), loop(), outputTelemetry(), updateOLED()
How is TMP117 raw data handled?	src/tmp117_protocol.h; readTmp117(), initTmp117(); diagnostics/tmp117_raw/src/main.cpp
How is the packet validated?	telemetry_bridge.py: parse_packet(), _validate_packet(), _validate_temperature(), _validate_fever()
How is history stored?	telemetry_store.py: measurements(), TelemetryStore.record(), TelemetryStore.history()
Where is temperature safety defined?	temperature_safety.py: evaluate_temperature_safety()
How does the baseline work?	personal_baseline.py: PersonalBaseline.summary(), .compare()
Is there AI?	ai_decision_engine.py: deterministic evaluate_clinical_status() only
Where are the exact dashboard labels?	assets/dashboard.js metric metadata; assets/dashboard_core.js specs

Question	Start here
How are gaps and CSV handled?	DashboardCore.gapAwarePath(), DashboardCore.csv(); tests/dashboard_core.test.js
What evidence exists?	README.md, CODE_AUDIT_MANIFEST.md, CODE_CLEANUP_REGISTER.md, NEXT_STEPS.md, tests/

19. One-sentence closing for the team

We built an engineering prototype that collects and validates multi-sensor telemetry, preserves missing data as missing, and refuses to turn an actual TMP117 sensor reading or an unverified wrist reading into a clinical core-temperature or fever claim.